Digitale shuttle: compact ontwerp

Onderbouwing, digitale kaart, vijf kernklassen en testplan | Versie 2.0 | 3 november 2026

Een haalbare opzet voor een ESP32-lijnvolger die bestemmingen via MQTT ontvangt, attracties in volgorde bezoekt en terugkeert naar het treindepot.

De scope in een zin

Een digitale graaf en routeplanner, een kleine ritbesturing op de robot en een eenvoudige logische simulatie om routes te controleren.

Wel bouwen	Buiten de eerste versie
Gehele kaart; richtingbewuste routeplanning	Natuurkundige simulatie van motoren, batterij en sensoren
Vijf kernklassen; bestaande functies hergebruiken	42 klassen, complexe interfaces of microservices
MQTT-opdrachten, wijziging onderweg, status	Meerdere shuttles, verkeersregeling en vlootbeheer
Gerichte softwaretests en fysieke kernritten	Automatisch herstel van iedere onbekende positie

Waarom dit nodig is

Losse lijsten met links/rechts worden kwetsbaar zodra vertrekpunt, bezoekvolgorde of terugweg verandert. Een kaart legt de aansluitingen eenmaal vast; de planner leidt de afslag af uit de actuele aankomstrichting. Dat vermindert routefouten zonder een uitgebreide simulator te vereisen.

Lees dit als ontwerp

Deze PDF vervangt voor de eerste bouwfase het eerdere uitgebreide ontwerp. De vijf klassen zijn een praktische indeling, geen verplichting van Python. De beschreven bestanden en tests zijn nog te bouwen. De huidige firmware is niet gewijzigd.

Wat de simulatie wel en niet bewijst

Een correcte digitale route bewijst de logica van het plan. Zij bewijst nog niet dat de echte sensoren elk kruispunt herkennen of dat de robot goed keert. Daarvoor blijven baanmetingen en fysieke tests nodig.

02 Leeswijzer en werkafspraken

Gebruik de compacte versie als bouwplan; extra onderdelen pas toevoegen bij een concrete behoefte.

Inhoud

Bronfoto van het parcours	3
Digitale topologie en vaste IDs	4
Puntenregister van het parcours	5
Verbindingenregister: hoofdwegen	6
Zijpaden en serviceplaatsen	7
Routeplanning met richting	8
Mappen en bestanden	9
Klassencontracten: kaart en rit	10
Klassencontracten: meting en MQTT	11
Volledig klassendiagram	12
Gegevens zonder extra klassen	13
Uitvoering in korte stappen	14
MQTT: klein en eenduidig contract	15
Wijzigen, herverbinden en afronden	16
Eenvoudige simulatie en integratie	17
Acceptatiecriteria	18
Testscenario's 01-06	19
Testscenario's 07-12	20
Testscenario's 13-18	21
Testuitvoering en bewijs	22
Bouwvolgorde en extra stappen	23
Open beslissingen en bronnen	24

Afspraken voor deze versie

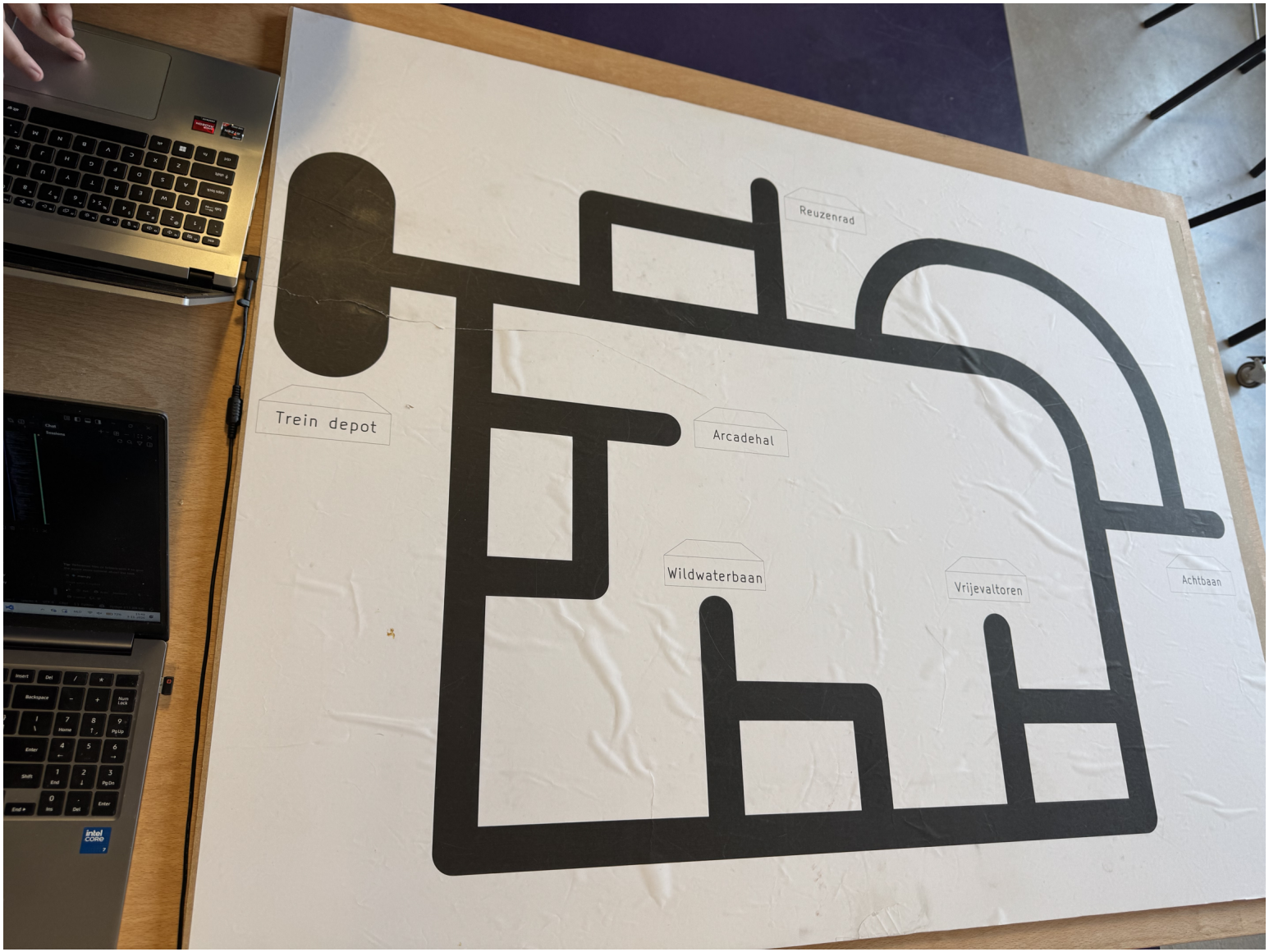
Een centrale afzender geeft opdrachten aan een robot. Per rit 1 t/m 5 verschillende attracties, in opgegeven volgorde, daarna depot. Geen automatische herschikking; dubbele doelen worden afgewezen. Per gevraagd bezoek eenmaal 5 seconden wachten.

Een wijziging vervangt de nog niet uitgevoerde bestemmingen. Reeds uitgevoerde bezoeken blijven in het ritlog. Het lopende bezoek wordt eerst afgerond. Onbereikbare of ongeldige opdrachten overschrijven geen werkend plan.

Acceptatiecriteria en toleranties hieronder zijn voorgestelde projectafspraken. Stem ze af op jullie opdracht/rubric; de hardwareafstelling wordt niet uit deze PDF afgeleid.

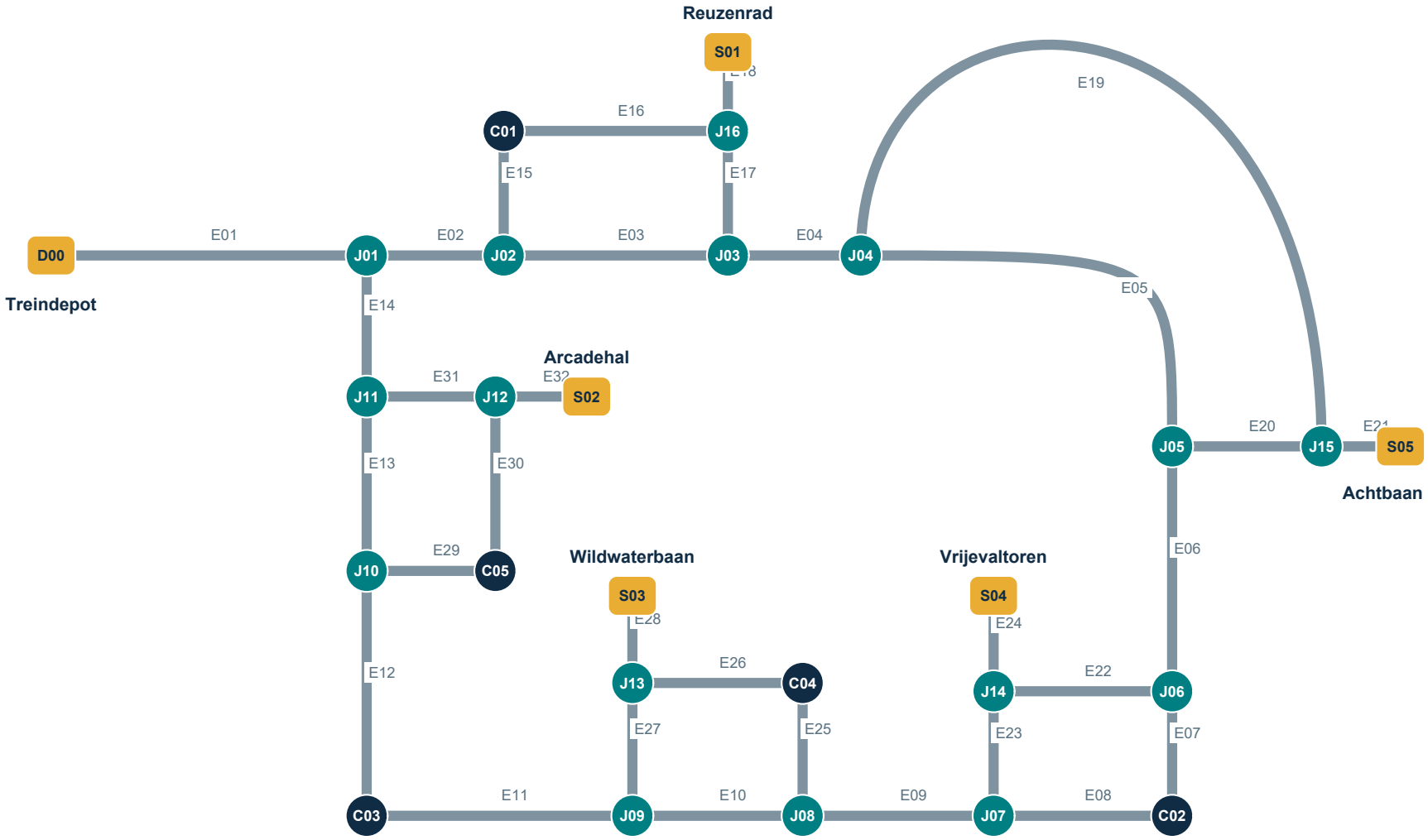
03 Bronfoto van het parcours

Aangeleverd bovenaanzicht, alleen voor leesrichting gedraaid. Geen lengtemeting of servicevalidatie.



04 Digitale topologie en vaste IDs

Schematische hertekening op basis van de foto. Lokale poorten zijn N/E/S/W in deze tekening; geen kompasmeting.



D = depot; J = keuzepunt; C = verplichte hoek; S = getekend eindpunt bij attractie. Een ronde doorgaande bocht is edge-geometrie, geen extra kruispunt.

27 punten | 32 verbindingen | 6 eindpunten | 6 lussen | 112 lokale overgangen. Service-/ankerplaatsen en werkelijke afstanden zijn nog te valideren.

05 Puntenregister van het parcours

Zelfde IDs als de schematische kaart. N/E/S/W zijn lokale poorten in de tekening, geen kompasmeting.

ID	Soort / label	Poorten
D00	Depot - Treindepot	E
J01	Keuzepunt - Depot-aansluiting	WES
J02	Keuzepunt - Reuzenrad-lus west	WEN
J03	Keuzepunt - Reuzenrad-stam	WEN
J04	Keuzepunt - Buitenbocht-keuze	WEN
J05	Keuzepunt - Achtbaan / rechterrand	NES
J06	Keuzepunt - Vrijeval-zijpad	NSW
J07	Keuzepunt - Vrijeval / onderrand	WEN
J08	Keuzepunt - Wildwater-zijpad / onderrand	WEN
J09	Keuzepunt - Wildwater-stam / onderrand	WEN
J10	Keuzepunt - Arcade-teruglus	NSE
J11	Keuzepunt - Arcade-inrit	NSE
J12	Keuzepunt - Arcade-T	WES
J13	Keuzepunt - Wildwater-T	NSE
J14	Keuzepunt - Vrijeval-T	NSE
J15	Keuzepunt - Achtbaan-T	NEW
J16	Keuzepunt - Reuzenrad-T	NSW
C01	Verplichte hoek - Hoek Reuzenrad-lus	SE
C02	Verplichte hoek - Hoek rechtsonder	NW
C03	Verplichte hoek - Hoek linksonder	NE
C04	Verplichte hoek - Hoek Wildwater-zijpad	SW
C05	Verplichte hoek - Hoek Arcade-lus	NW
S01	Getekend eindpunt - Reuzenrad	S
S02	Getekend eindpunt - Arcadehal	W
S03	Getekend eindpunt - Wildwaterbaan	S
S04	Getekend eindpunt - Vrijevaltoren	S
S05	Getekend eindpunt - Achtbaan	W

Ronde doorgaande bochten zitten in de vorm van een verbinding. Een C-punt is een expliciete haakse hoek met maar een doorgaande uitgang, geen extra keuze.

06 Verbindingenregister: hoofdwegen

Alle 32 verbindingen zijn in dit ontwerp tweerichtingsverkeer. Dit moet het team op de echte baan bevestigen.

ID	Van / poort	Naar / poort	Opmerking
E01	D00 / E	J01 / W	Recht / hoek gekoppeld via punt
E02	J01 / E	J02 / W	Recht / hoek gekoppeld via punt
E03	J02 / E	J03 / W	Recht / hoek gekoppeld via punt
E04	J03 / E	J04 / W	Recht / hoek gekoppeld via punt
E05	J04 / E	J05 / N	Binnenbocht
E06	J05 / S	J06 / N	Recht / hoek gekoppeld via punt
E07	J06 / S	C02 / N	Recht / hoek gekoppeld via punt
E08	C02 / W	J07 / E	Recht / hoek gekoppeld via punt
E09	J07 / W	J08 / E	Recht / hoek gekoppeld via punt
E10	J08 / W	J09 / E	Recht / hoek gekoppeld via punt
E11	J09 / W	C03 / E	Recht / hoek gekoppeld via punt
E12	C03 / N	J10 / S	Recht / hoek gekoppeld via punt
E13	J10 / N	J11 / S	Recht / hoek gekoppeld via punt
E14	J11 / N	J01 / S	Recht / hoek gekoppeld via punt
E15	J02 / N	C01 / S	Recht / hoek gekoppeld via punt
E16	C01 / E	J16 / W	Recht / hoek gekoppeld via punt
E17	J16 / S	J03 / N	Recht / hoek gekoppeld via punt
E18	J16 / N	S01 / S	Recht / hoek gekoppeld via punt

Wat een verbinding betekent

Een verbinding is een berijdbare lijn tussen twee geregistreerde punten. De lokale poorten bepalen de rijrichting bij beide uiteinden. Vooral bij de ronde buitenbocht mag de planner niet alleen het verschil tussen de getekende x/y-coordinaten gebruiken.

Werkelijke afstanden zijn nog niet ingemeten. De eerste planner minimaliseert het aantal verbindingen, niet de reistijd of meters. De schetscoordinaten zijn alleen voor de weergave.

07 Zijpaden en serviceplaatsen

Een attractienaam, een getekend lijneinde en een echt stoppunt zijn verschillende gegevens.

ID	Van / poort	Naar / poort
E19	J04 / N	J15 / N
E20	J15 / W	J05 / E
E21	J15 / E	S05 / W
E22	J06 / W	J14 / E
E23	J14 / S	J07 / N
E24	J14 / N	S04 / S
E25	J08 / N	C04 / S
E26	C04 / W	J13 / E
E27	J13 / S	J09 / N
E28	J13 / N	S03 / S
E29	J10 / E	C05 / W
E30	C05 / N	J12 / S
E31	J12 / W	J11 / E
E32	J12 / E	S02 / W

Serviceprofiel: eerst vastleggen, dan op de robot gebruiken

Opdracht-ID	Attractie	Logisch eindpunt	Kandidaat aansluiting
FERRIS	Reuzenrad	S01	J16
ARCADE	Arcadehal	S02	J12
WILDWATER	Wildwaterbaan	S03	J13
FREEFALL	Vrijevaltoren	S04	J14
COASTER	Achtbaan	S05	J15

Logisch zijn S01 t/m S05 de doelen. De huidige robotcode gebruikt een T-halte; fysiek kan het stoppunt op de aansluiting liggen. Kies voor de eerste fysieke versie per attractie precies een gevalideerd paar (aankomstedge, vertrekedge), plus doelknoop en aankomstherkenning. De planner moet die aanrit zoeken en de vertrekedge afdwingen; alleen S03 wijzigen naar J13 is onvoldoende. Wijzig de kaartversie en test opnieuw.

Geen stille vervanging

S03 is het getekende Wildwatereindpunt; J13 is de aansluiting. Die mogen niet ongemerkt dezelfde plaats worden. Een U-bocht op S03 en een afslag op J13 leveren een andere terugweg op.

08 Routeplanning met richting

Een route is een reeks aansluitende bewegingen en bezoeken, geen lijst met losse afslagnummers.

Eenvoudige aanpak

- Start met een bekende knoop en inkomende verbinding; vanuit depot start de robot richting J01.

- Gebruik BFS op de toestand (knoop, inkomende verbinding). Elke verbinding heeft eerst kost 1. Sorteer verbindingen op ID voor een vaste keuze bij gelijke paden.

- Plan achtereenvolgens naar elk doel en daarna D00. Een serviceprofiel kan ook de aankomstedge eisen. Dwing de vastgelegde vertrekedge af voordat je de volgende etappe verder plant.

- Voeg SERVICE na elke gevraagde aankomst toe, en END bij het depot. Door een servicepunt rijden is niet vanzelf een bezoek.

Geen directe U-bocht op gewone knopen. Bij terminal/depot of een expliciet gevalideerde serviceovergang mag een geplande keeractie wel. Uitgeschakelde verbindingen zijn uitgesloten. Verplichte hoeken hebben een eigen hoekstap; een doorgaande bocht volgt de lijn. Onbereikbaar via de vereiste aanrit betekent afwijzen, geen illegale uitgang kiezen.

Voorbeeld bij J01	Resultaat
Inkomend E01, uitgaand E14	Rechts
Inkomend E02, uitgaand E14	Links

Gecontroleerd voorbeeld: alleen het logische eindpuntprofiel

D00 -> S03: D00 - J01 - J11 - J10 - C03 - J09 - J13 - S03

S03 -> S04: S03 - J13 - C04 - J08 - J07 - J14 - S04

S04 -> D00: S04 - J14 - J06 - J05 - J04 - J03 - J02 - J01 - D00

Dit voorbeeld demonstreert de kaartlogica en schrijft niet voor waar de echte robot moet keren. Het fysieke serviceprofiel bepaalt de definitieve vertrekbewegingen.

09 Mappen en bestanden

Een klein Python-project met gedeelde kaartlogica. De robot blijft MicroPython; de simulator draait op een gewone computer.

```
shuttle/
  gedeeld/        # kaart en planner; ook op ESP32
  data/           # parcours.json
  robot/          # besturing en bestaande hardwarecode
  simulatie/      # eenvoudige weergave op PC
  tests/          # gerichte tests op PC
  README.md
  requirements-sim.txt
```

Bestand(en)	Verantwoordelijkheid
gedeeld/parcours_kaart.py	ParcoursKaart; kaartvalidatie en lokale afslagregels.
gedeeld/route_planner.py	RoutePlanner; route en service-/eindstappen opbouwen.
data/parcours.json	Punten, verbindingen, poorten, kaartversie en logische serviceplaatsen.
robot/main.py	Maakt objecten en voert de korte hoofdloop uit; geen Main-klasse.
robot/rit_besturing.py	RitBesturing; actieve opdracht, toestand, voortgang en plaatsinschatting.
robot/kruispunt_detector.py	KruispuntDetector; bevestiging, vrijgave en passagevolgnummer.
robot/robot_mqtt.py	RobotMqtt; bestaande client/fabrieksfunctie behouden waar bruikbaar.
robot/lijnvolger.py	Bestaande PID-, zoek- en draaifuncties; geen extra klasse verplicht.
robot/hardware.py	Bestaande Pin/PWM-, motor- en leesfuncties; alleen ESP32 importeert machine.
robot/objectdetectie.py	Bestaande ObjectDetectie en afstandsmeting; niet opnieuw ontwerpen.
robot/config.py	Pinnen, polariteit, afstelling en gevalideerde service-/depotprofielen.
simulatie/main.py	Logische route-uitvoering en eenvoudige weergave; CPython, geen machine-import.
tests/test_kaart_planner.py	T01-T05 en richtingbewuste referentiecontrole.
tests/test_besturing_detector.py	T09-T16, toestanden en bestuurbare klok.
tests/test_mqtt_opdrachten.py	T06-T08 en T17 met een namaakclient.
README.md / requirements-sim.txt	Starten, testen, MQTT-contract en uitsluitend PC-afhankelijkheden.

Voeg waar nodig lege __init__.py-bestanden toe voor packages. Zet gedeeld/, data/ en de benodigde robotmodules op de ESP32; de simulator en PC-pakketten horen daar niet. main.py op de ESP32 start de robot. Bewaar werkende firmware als terugvalversie.

10 Klassencontracten: kaart en rit

Een dict of list blijft gewone data. Alleen de vijf kernobjecten krijgen een eigen klasse.

ParcoursKaart | gedeeld/parcours_kaart.py

Beheert de hele baan en de gekozen serviceplaatsen.

Gegevens / toestand	Methoden (ontwerpcontract)
versie: str knopen: dict verbindingen: dict attracties: dict incl. serviceprofiel geblokkeerd: set	valideer() -> list buren(knoop_id) -> list beweging(knoop, in_edge, uit_edge) -> str doelknoop(attractie_id) -> str

RoutePlanner | gedeeld/route_planner.py

Berekent een aaneengesloten plan met de juiste aankomstrichting.

Gegevens / toestand	Methoden (ontwerpcontract)
kaart: ParcoursKaart	plan(positie, bestemmingen) -> dict pad(start, in_edge, doel, doel_in=None) -> list valideer_plan(plan) -> list

RitBesturing | robot/rit_besturing.py

Beheert de actieve rit, bestuurt bestaande functies en verwerkt voortgang.

Gegevens / toestand	Methoden (ontwerpcontract)
planner: RoutePlanner detector: KruispuntDetector mqtt: RobotMqtt toestand: str; hervat_toestand: str positie: dict; opdracht: dict plan: dict; stapindex: int bezoeken: list; wacht_sinds_ms: int | None wachtend: dict | None laatste_passage_seq: int; status_seq: int	ontvang(opdracht) -> dict update(nu_ms, meting) -> None verwerk_waarneming(waarneming) -> None pas_wachtende_opdracht_toe() -> None status() -> dict

11 Klassencontracten: meting en MQTT

Bestaande functies en modules behouden waar ze werken; alleen de verantwoordelijkheden expliciet scheiden.

KruispuntDetector | robot/kruispunt_detector.py

Filtert ruwe metingen en meldt hoogstens een punt per passage.

Gegevens / toestand	Methoden (ontwerpcontract)
vrijgegeven: bool kandidaat_sinds_ms: int | None lijn_sinds_ms: int | None passage_seq: int instellingen: dict	update(nu_ms, patroon) -> dict | None bevestig_verlaten() -> None reset() -> None

RobotMqtt | robot/robot_mqtt.py

Hergebruikt de MQTT-client voor opdrachten, bevestigingen en status.

Gegevens / toestand	Methoden (ontwerpcontract)
client: bestaand clientobject robot_id: str; sessie_id: str inbox: list; recente_opdrachten: dict hoogste_command_seq: int status_wachtend: dict | None	poll() -> None pak_opdracht() -> dict | None bevestig(command_id, resultaat, reden) -> None publiceer_status(status) -> None

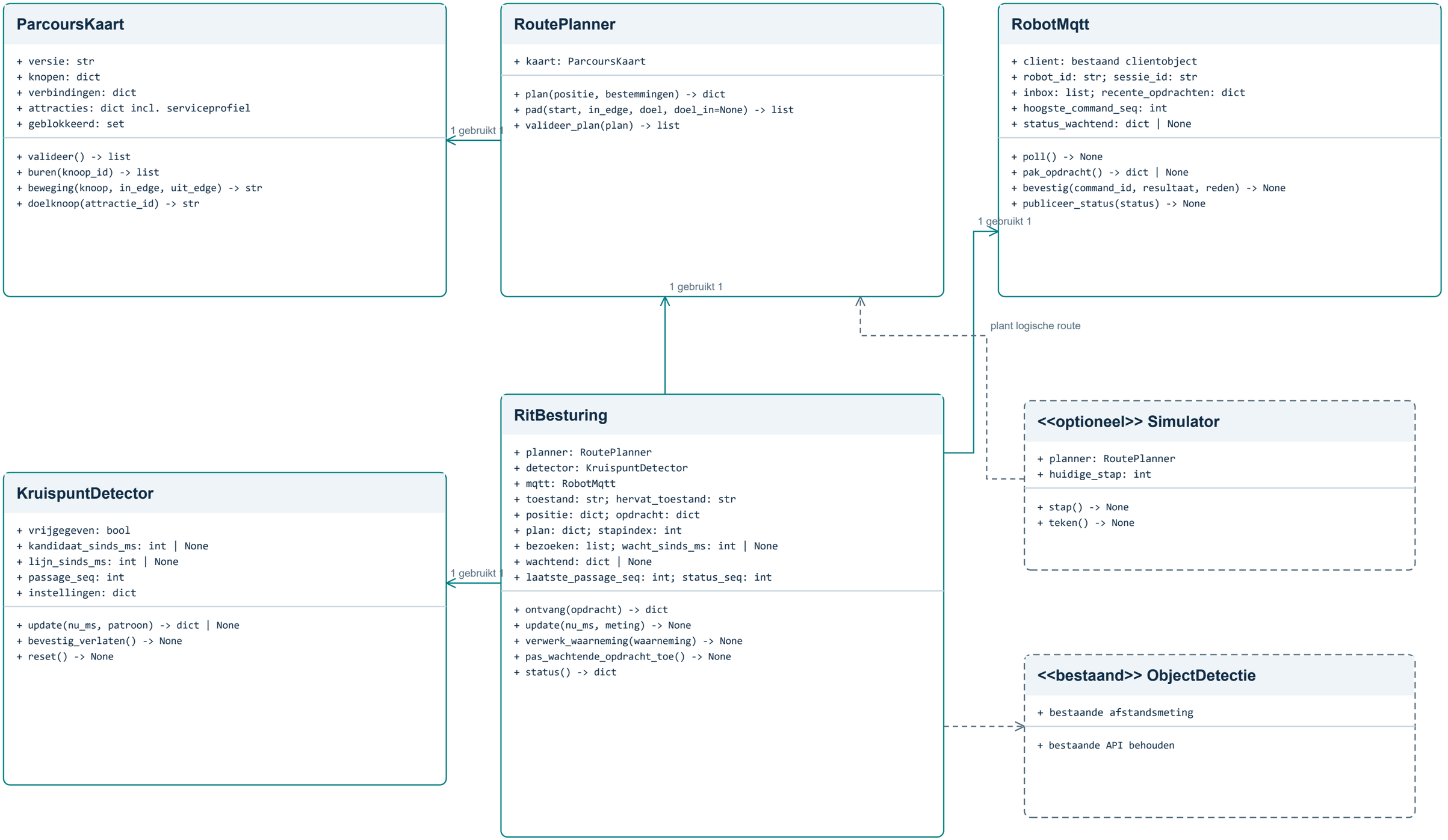
Wat geen nieuwe klasse hoeft te worden

Onderdeel	Keuze in deze scope
Knoop / verbinding / attractie	Dictionary in ParcoursKaart.
Opdracht / plan / stap / positie	Dictionary of lijst; valideren met functies.
PID, lijn volgen, draaien, zoeken	Functies in lijnvolger.py. Langlopende acties later in korte updates uitvoeren.
GPIO / PWM / sensoren	Bestaande functies in hardware.py.
Obstakeldetectie	Bestaande ObjectDetectie; de echte API eerst controleren.
Kaart tekenen / testklok	Functies; geen verplichte interface- of klokklasse.
Simulator	Optioneel een kleine klasse; eerst mag simulatie/main.py functies gebruiken.

De genoemde methoden zijn ontwerpcontracten. De exacte API van robot_mqtt.py en objectdetectie.py kon in deze werkmap niet worden gecontroleerd omdat deze bestanden ontbreken. Bij implementatie eerst aansluiten op de bestaande modules.

12 Volledig klassendiagram

Vijf verplichte kernklassen. Stippelvakken zijn bestaande onderdelen of optionele uitbreiding. + = publiek; pijl = gebruikt.



De planner verwijst naar een kaart; RitBesturing gebruikt een planner, detector en MQTT-object. Optionele Simulator gebruikt de planner en diens kaart. PID, motor- en sensorfuncties zijn modules en dus geen klassen in dit diagram. main.py maakt de objecten. Geen overerving nodig.

13 Gegevens zonder extra klassen

Voorbeeldstructuren; dezelfde velden krijgen op PC en ESP32 dezelfde betekenis.

```
knoop = {"id": "J01", "soort": "keuze", "poorten": "WES"}
verbinding = {"id": "E14",
  "a": "J01", "a_poort": "S",
  "b": "J11", "b_poort": "N", "actief": True}
pos = {"laatste_punt": "J01", "in_edge": "E01",
  "verwacht_punt": "J11", "status": "route_aannemelijk"}
stap = {"van": "J01", "via": "E14", "naar": "J11",
  "soort": "keuze", "actie": "rechts"}
service = {"soort": "service", "doel": "WILDWATER",
  "knoop": "S03", "wacht_ms": 5000}
einde = {"soort": "einde", "knoop": "D00"}
```

Regels die de code moet controleren

- Unieke IDs; elke verbinding verwijst naar twee bestaande punten. Elke eindpoort wordt eenmaal gebruikt; kaartversie is verplicht.

- Een afslag hoort bij de van-knoop. Eerst de passage en juiste uitgang verwerken; dan pas de volgende stap actief maken. Bewaar ook welke verbinding nu wordt bereden.

- Een service wordt eenmaal gemarkeerd na aankomst en wachten. De daaropvolgende vertrekstap wordt actief voordat vertrekpatronen worden verwerkt.

- Gebruik aparte tellers: passage_seq voor detectorpassages, command_seq voor opdrachten en status_seq voor uitgaande status. RitBesturing negeert een al verwerkte passage_seq.

- Bij een nieuwe rit: plan, stapindex, detector en laatste_passage_seq samen resetten, evenals bezoeklijst, PID-historie en wachtgegevens. command_seq blijft sessiebreed behouden. Een obstakel behoudt de onafgeronde stap.

Positie is een inschatting

Een detector ziet een patroon, niet het bordje J01. Een bekende start en betrouwbare telling geven een bruikbare route-inschatting. Na gemiste of strijdige waarnemingen wordt de positie onzeker; geen routewissel of fictief succes. Extra markeringen zijn een mogelijke latere verbetering.

14 Uitvoering in korte stappen

RitBesturing is een kleine toestandsmachine; geen nieuwe klasse per toestand.

Toestand	Gedrag en overgang
GEREED	Motoren stil. Accepteer een geldige start vanuit bevestigde startplaats.
RIJDEN	Bestaande lijnvolgfunctie; detector meldt passage. Kies geplande uitgang.
DRAAIEN	Voer bocht/keeractie uit in updates; bevestig verlaten en juiste uitgaande lijn.
WACHTEN	Eenmaal 5 s actieve haltetijd; MQTT blijven verwerken. Daarna volgende etappe.
ZOEKEN	Lijn zoeken zonder maximale zoektijd. Geen stap verbruiken; onzekerheid melden.
GEBLOKKEERD	Obstakel: motoren stil; onthoud vorige toestand. Vrijgave hervat onafgeronde actie.
GESTOPT	Expliciete stop of een onuitvoerbare/strijdige situatie die veilig ingrijpen vraagt. Geen automatisch succes.
KLAAR	Alle diensten afgerond en depot bevestigd. Verstuur succesvol; gereed voor volgende rit.

Korte hoofdloop

```
meting = lees_sensoren()
besturing.update(nu_ms(), meting)
mqtt.poll()                # kort, geen wacht op bericht
opdracht = mqtt.pak_opdracht()
if opdracht is not None:
    besturing.ontvang(opdracht)
# Status alleen op verandering of afgesproken interval.
```

Geen sleep(5) voor een halte. Met MicroPython tijdsverschillen via ticks_diff berekenen, niet via gewone aftrekking [B2]. Bestaande blokkerende draai-/zoeklussen worden stapsgewijs omgezet; alleen in een klasse stoppen maakt ze niet niet-blokkerend.

Detecteren bij rechtdoor passeren veroorzaakt geen opzettelijke meetstop of lagere kruissnelheid. Echte bochten, haltes, obstakels en expliciet stoppen mogen de motoractie wel veranderen.

15 MQTT: klein en eenduidig contract

Voorgesteld contract. Brokeradres en bestaande client worden bij implementatie aangesloten; geen extra serverframework nodig.

Topic (voorbeeld)	Gebruik / retain
school/shuttle/robot01/command	Opdracht, bevestiging op applicatieniveau nodig; retain uit.
school/shuttle/robot01/ack	accepted / pending / applied / rejected / duplicate; retain uit.
school/shuttle/robot01/status	Laatste toestand en laatste ritresultaat; retain mag aan, met sessie en seq.

```
{
  "command_id": "cmd-001", "command_seq": 1,
  "robot_id": "robot01", "sessie_id": "boot-a7f2",
  "actie": "start", "mission_id": "rit-001", "revisie": 1,
  "kaartversie": "foto-v1-logisch",
  "bestemmingen": ["WILDWATER", "FREEFALL"]
}
```

De status bij opstart meldt de nieuwe sessie_id. De centrale afzender gebruikt die waarde en oplopende command_seq. Een start is alleen geldig vanuit GEREED/KLAAR of na gecontroleerd herstel. Onderweg is actie vervang nodig; een tweede start wordt afgewezen.

De firmware plant lokaal met de kleine gedeelde planner; MQTT draagt bestemmingen, geen PWM-commando's. Maximaal vijf unieke bekende doelen; dezelfde kaartversie is vereist. actie stop mag de rit onderbreken: dezelfde sessie, een nieuw ID en volgnummer, geen bestemmingen nodig. Alleen een nieuwe gevalideerde start na positiebevestiging verlaat GESTOPT.

Ontvangen is nog niet gelukt

accepted betekent opdracht geaccepteerd; pending betekent wachten op een geschikte grens; applied betekent werkelijk actief gemaakt. Alleen status.resultaat = succesvol bewijst dat de rit volgens de afgesproken controles is afgerond.

MQTT QoS 1 kan dubbele afleveringen geven [B1]. Gebruik indien de bestaande client dit goed ondersteunt QoS 1 voor opdrachten/ACK. Anders zijn retries met dezelfde opdracht-ID en applicatiebevestiging nodig; dat pad moet apart getest worden. Geen exactly-once-garantie claimen.

16 Wijzigen, herverbinden en afronden

Deze regels blijven binnen RitBesturing en RobotMqtt. Geen aparte coördinator, database of hersteldienst vereist.

Wijziging tijdens een rit

- De actie vervang houdt dezelfde mission_id en heeft een hogere revisie. Valideer eerst inhoud, sessie, volgnummer en kaartversie; pas daarna de wachtende opdracht aan.

- Accepteer alleen een revisie hoger dan zowel de actieve als de wachtende revisie. De nieuwste geldige vervangt de wachtende versie. Een foute nieuwere opdracht wist haar niet.

- Pas toe op de volgende geschikte herkende knoop, voordat de uitgang gekozen wordt, met bruikbare plaats- en richtingsinschatting. Tijdens DRAAIEN ontvangen: eerst de bocht afmaken, dan wachten op de volgende knoop. Niet terugplannen vanaf het al verlaten punt.

- Een lopend servicebezoek wordt inclusief de 5 seconden afgerond. De wijziging vervangt alleen de daarna resterende doelen; al bediende doelen opnieuw aanvragen wordt in versie 1 afgewezen.

- Is de plaats onzeker, houd de wijziging wachtend en meld dat. Na opnieuw bevestigen opnieuw plannen; oude en nieuwe stappenlijsten niet combineren.

Dubbele of oude opdrachten

RobotMqtt bewaart de laatste 32 geaccepteerde command_id/resultaat-combinaties en het hoogste geaccepteerde command_seq van de huidige sessie. Een oudere seq buiten het geheugen wordt als verouderd afgewezen. Daardoor kan een vergeten oude ID geen nieuwe rit starten. Een ID met gewijzigde inhoud wordt afgewezen. De centrale afzender beheert de volgnummers.

Verbinding weg en herstart

Bij netwerkverlies mag een lokaal geldig plan doorgaan als routecontext en obstakels dat toelaten. Laatste status in RAM bijwerken; bij herverbinden dezelfde ritstatus opnieuw publiceren. ACK-verlies leidt tot retry met dezelfde ID. Geen opdracht als retained bericht laten klaarstaan.

Na stroomreset: motoren stil, nieuwe sessie, oude opdrachten ongeldig en geen automatisch hervatten. Startplaats opnieuw bevestigen. Volledig herstel na stroomuitval en permanente gebeurtenisopslag vallen buiten deze versie.

```
{"robot_id":"robot01", "sessie_id":"boot-a7f2",
 "mission_id":"rit-001", "revisie":1, "status_seq":37,
 "bron":"robot", "toestand":"KLAAR",
 "bezocht":["WILDWATER","FREEFALL"],
 "positie_status":"depot_bevestigd",
 "resultaat":"succesvol"}
```

17 Eenvoudige simulatie en integratie

De eerste simulatie is een test van de kaart en ritlogica. Geen nagebootste fotorealistische of natuurkundige robot.

Wat je op de computer ziet

De volledige kaart, een robotmarkering, bestemmingen in volgorde, actieve verbinding, geplande volgende beweging en toestand. Bedieningen: kies doelen, start, stap, pauze en reset. Simulatieberichten krijgen bron = simulatie en een aparte robot_id/topicruimte.

Een eenvoudige uitvoering loopt over de berekende route en verwerkt service-/eindstappen. Tekenfuncties en een bestuurbare klok zijn voldoende. Een Simulator-klasse is pas nodig als de simulatie zelf veel toestand krijgt. Er is geen nieuwe backend, database of webframework vereist.

Twee verschillende controles

Controle	Wat hiermee aangetoond wordt
Logische simulatie	Verbindingen, richting, volgorde, service, terugkeer en wijzigingen op routepunten.
Besturingstest met nepmetingen	RitBesturing en KruispuntDetector krijgen meetreeksen en tijd van de test. Motorfuncties worden vervangen door registrerende testfuncties.
Fysieke proef	Of echte sensoren, draaipunt, snelheid, halteprofiel en depotmethode betrouwbaar werken.

Integreren zonder alles tegelijk te herschrijven

- Bewaar een werkende firmwareversie. Leg de huidige pinnen, vijf sensorbits, zwart/wit-polariteit en motorpolariteit vast.

- Begin alleen met kaart en planner op PC. Voeg detector- en besturingstests toe voordat je vaste routegegevens vervangt.

- Hergebruik PID-, motor- en objectfuncties. Maak hun uitvoering kort genoeg om communicatie en metingen te blijven verwerken.

- Test eerst een korte fysieke etappe en terugkeer; daarna kernrit, alle attracties en wijziging onderweg.

Batterij en snelheid

De eerdere waarde basis = 0.35 is een gerapporteerde werkende afstelling, geen garantie bij iedere batterijspanning. Meet de werkelijke spanning en sensorpassages in twee representatieve voedingscondities. Pas timing of snelheid alleen op basis van die metingen aan.

18 Acceptatiecriteria

Voorgestelde, toetsbare afspraken voor de compacte oplevering. Nog geen testresultaten of fysieke vrijgave.

ID / onderwerp	Wanneer akkoord?	Test
AC01 Kaart klopt	27 punten en 32 verbindingen uit de foto zijn geregistreerd. Alle verwijzingen en poorten zijn geldig; team vergelijkt tekening en baan.	T01
AC02 Richtingbewust plannen	Een plan sluit aan, houdt rekening met aankomstrichting en het serviceprofiel, of meldt expliciet dat geen pad bestaat.	T02, T03
AC03 Bezoekvolgorde	Aangevraagde attracties worden in exact de ontvangen volgorde bediend. Langs een attractie rijden geldt niet als bezoek.	T04, T05, T18
AC04 Terug naar depot	Elk plan eindigt met een depottraject en een aparte eindcontrole. Een lege stappenlijst is onvoldoende voor succesvol.	T04, T05, T14
AC05 Een passage, een melding	Een kruispunt levert maximaal een passage-event. Na bevestigd verlaten kan de volgende passage opnieuw worden gemeld.	T09, T11
AC06 Hoek en afslag scheiden	Een verplichte hoek gebruikt zijn eigen planstap. De opdracht voor een latere afslag wordt niet daardoor verbruikt.	T10, T11
AC07 Geen herhaalde halte	Een afgeronde stap en een uitgevoerd bezoek worden niet opnieuw uitgevoerd door vertrek- of keerpatronen.	T04, T12, T16, T18
AC08 Eenmaal vijf seconden	Vertrek niet voor 5.000 ms actieve haltetijd en in de kloktest uiterlijk twee updates later. Fysiek voorstel: 5,0 +/- 0,5 s.	T13
AC09 Ongeldig bericht afwijzen	Foute JSON, onbekende doelen of een ongeldige sessie/revisie geven een afwijzing met reden. Een geldige lopende rit blijft intact.	T06
AC10 Dubbele opdracht herkennen	Hetzelfde command_id start binnen dezelfde sessie hoogstens een uitvoering. Ook een retry na aankomst herhaalt de rit niet.	T07, T17
AC11 Wijziging onderweg	Een geldige wijziging wordt wachtend gezet en pas op een geschikt routepunt toegepast. Een bocht wordt niet halverwege vervangen.	T08, T13
AC12 Blijven zoeken	Een zoektijd alleen beeindigt de rit niet. Zoeken verbruikt geen stap; expliciet stoppen en obstakelblokkering houden voorrang.	T15, T16
AC13 Netwerk en herstart	Na herverbinden blijft de actieve rit dezelfde. Na een herstart start de robot niet automatisch uit een onbevestigde positie.	T17
AC14 Succes met bewijs	Succes volgt pas na alle bezoeken en depotbevestiging. Fysieke vrijgave: voorgesteld vijf correcte kernritten per twee vastgelegde voedingscondities.	T14, T18

19 Testscenario's 01-06

Elk scenario vastleggen met versie, invoer, werkelijk resultaat, bewijs en geslaagd/mislukt. Alle scenario's staan nog op gepland.

T01 | Kaart en foute verwijzingen | AC01

Opstelling: Kaart/foto + validatiefunctie.

Actie: Vergelijk alle punten en verbindingen. Verwijder in een kopie een referentieknoop; verbreek apart het depotpad.

Verwacht: Geldige kaart slaagt; ontbrekende verwijzing en onbereikbaar doel krijgen een concrete fout. Bewijs: afgevinkte kaart en validatielog.

T02 | Alle richtingbewuste starttoestanden | AC02

Opstelling: Geautomatiseerde plannercheck op de logische kaart.

Actie: Test alle knopen met iedere inkomende verbinding tegen ieder doel. Test ook een geblokkeerd doel en een fysiek profiel met vaste aanrit/vertrekedge.

Verwacht: Pad respecteert richting en serviceprofiel; bij onbereikbaarheid volgt een fout. Vergelijk het pad met een onafhankelijke referentie.

T03 | Zelfde uitgang, andere aankomstrichting | AC02

Opstelling: J01 met inkomend E01 respectievelijk E02.

Actie: Vraag vanuit beide richtingen de uitgang via E14 aan.

Verwacht: Vanuit E01 is dit rechts; vanuit E02 is het links. Geen afslagtekst uitsluitend op basis van doelknoop.

T04 | Kernrit zonder herbezoek | AC03, AC04, AC07

Opstelling: Logische kaart; begin D00 richting J01.

Actie: Vraag Wildwaterbaan, daarna Vrijevaltoren. Voer het hele plan inclusief terugweg uit.

Verwacht: Servicevolgorde S03, S04, D00 in het logische eindpuntprofiel. Geen tweede Wildwaterbezoek. Bewijs: stappenlog en bezoeklijst.

T05 | Verschillende missievolgorden | AC03, AC04

Opstelling: Logische kaart en lijst van vijf attracties.

Actie: Test alle 325 geordende selecties van 1 t/m 5 verschillende attracties, gevolgd door depot.

Verwacht: Volgorde blijft gelijk; alle plannen sluiten aan. Een ongeplande passage langs een attractie telt niet als service.

T06 | Ongeldige opdrachten | AC09

Opstelling: Lopende geldige rit; berichteninjectie.

Actie: Stuur kapotte JSON, onbekend doel, meer dan vijf doelen, dubbele doelen, oude revisie en een verkeerde sessie.

Verwacht: Alle berichten worden afgewezen met reden; actief plan en stapindex blijven intact.

20 Testscenario's 07-12

Elk scenario vastleggen met versie, invoer, werkelijk resultaat, bewijs en geslaagd/mislukt. Alle scenario's staan nog op gepland.

T07 | Hetzelfde bericht meerdere keren | AC10

Opstelling: Een opdracht met vast command_id.

Actie: Lever driemaal: voor vertrek, onderweg en na aankomst. Herhaal ook na meer dan 32 andere geaccepteerde opdrachten.

Verwacht: Een uitvoering. Oude ID wordt na vol geheugen afgewezen of herkend; nooit als nieuwe rit behandeld. ACK en eindresultaat zijn verschillende berichten.

T08 | Wijziging tijdens draaien | AC11

Opstelling: Actieve rit revisie 1; robot in toestand DRAAIEN.

Actie: Stuur vervanging revisie 2, dan 3; daarna opnieuw revisie 2 met een hoger command_seq en een ongeldige revisie 4.

Verwacht: Bocht blijft intact. Revisie 3 blijft wachtend en gaat pas bij de volgende geschikte knoop in. Oude 2 en foute 4 verdringen haar niet.

T09 | Breed patroon en vrijgave | AC05

Opstelling: Opgenomen of synthetische metingen met tijdstempels.

Actie: Houd breed patroon vast, voeg korte ruis toe, bevestig gewone lijn en toon tweede kruispunt.

Verwacht: Een event voor eerste passage en een voor tweede; geen dubbel event door langdurig zwart.

T10 | Verplichte hoek voor keuze | AC06

Opstelling: Plan met C03 en daarna J09 of een gelijkwaardige route.

Actie: Volg de hoek en passeer daarna een keuzepunt.

Verwacht: De hoekstap wordt afgerond; de aparte opdracht op het keuzepunt blijft beschikbaar en wordt daar eenmaal uitgevoerd.

T11 | Korte en scheve afslag | AC05, AC06

Opstelling: Echte sensoropnames bij twee vastgelegde voedingscondities.

Actie: Neem korte aftakkingen en scheve benaderingen op. Speel de ruwe reeks terug en vergelijk met gemarkeerde passages.

Verwacht: Binnen het afgesproken meetbereik nul gemiste of dubbele testpassages. Geen opzettelijke meetstop bij rechtdoor passeren. Grenzen worden genoteerd.

T12 | Attractie verlaten | AC07

Opstelling: Gevalideerd service- en vertrekprofiel per attractie.

Actie: Kom aan, wacht en vertrek. Bied tijdens vertrek opnieuw een breed patroon van dezelfde aansluiting aan.

Verwacht: Volgende etappe blijft actief; geen tweede halte en geen terugzetting van de stapindex.

21 Testscenario's 13-18

Elk scenario vastleggen met versie, invoer, werkelijk resultaat, bewijs en geslaagd/mislukt. Alle scenario's staan nog op gepland.

T13 | Wachten terwijl MQTT actief blijft | AC08, AC11

Opstelling: Bestuurbare testklok; updateperiode 10 ms.

Actie: Controleer op 4.990, 5.000 en 5.020 ms. Lever tijdens wachten een wijziging. Test ook obstakelpauze.

Verwacht: Niet te vroeg vertrekken; maximaal twee updates vertraging zonder blokkade. MQTT blijft reageren. Pauzetijd telt niet mee als actieve haltetijd.

T14 | Geen vals depotresultaat | AC04, AC14

Opstelling: Actieve heenweg en aparte terugwegvariant.

Actie: Toon onderweg een depotachtig breed patroon. Laat op terugweg ook de lijst leeglopen zonder depotbevestiging.

Verwacht: Geen succesvol. Pas passende depotstap plus afgesproken herkenningsbewijs en alle bezoeken leveren een eindresultaat.

T15 | Lijn lang kwijt en later terug | AC12

Opstelling: Actieve rit, lijnverlies en later passende sensorreeks.

Actie: Simuleer 10 minuten lijnverlies, langer dan de oude grens, en bied daarna de lijn terug aan.

Verwacht: Blijft zoeken, verwerkt MQTT en behoudt routestap. Alleen normale hervatting bij voldoende routecontext; bij onzekere plek blijft onzeker-status.

T16 | Obstakel en expliciete stop | AC07, AC12

Opstelling: Objectdetectie ingeschakeld; lijnvolgen en zoeken.

Actie: Plaats obstakel, verwijder het en stuur apart een stopopdracht.

Verwacht: Motoren stoppen bij blokkade. Vrijgave hervat dezelfde onafgeronde actie; expliciete STOP blijft staan tot een geldige nieuwe start.

T17 | Verbinding weg of stroomreset | AC10, AC13

Opstelling: Actieve rit met vastgelegde mission_id en sessie.

Actie: Onderbreek netwerk en verbind opnieuw. Herstart in aparte variant de ESP32 en stuur daarna een oud bericht.

Verwacht: Netwerk: dezelfde rit, geen dubbel bezoek. Reset: nieuwe sessie, motoren stil, oude sessie afgewezen; opnieuw startpositie bevestigen.

T18 | Fysieke vrijgave en tweede rit | AC03, AC07, AC14

Opstelling: Baan gecontroleerd; twee gemeten voedingscondities; servicepunten vastgelegd.

Actie: Rijd kernrit vijfmaal per conditie. Rijd daarna ook de vijf attracties en start een tweede losse opdracht.

Verwacht: Alle bezoeken en terugkeer correct; halte 5 s binnen tolerantie; geen meetstop op doorgangen. Nieuwe opdracht heeft schone stap-/bezoekgegevens. Bewijs: video, spanning en ritlog.

22 Testuitvoering en bewijs

Houd ontwerpcontrole, softwaretests en fysieke acceptatie apart.

Fase	Uitvoeren / bewaren
Kaartreview	Vergelijk foto, schets en echte baan. Team tekent kaartprofiel af; service-/depotplaatsen worden vastgelegd.
PC-tests	Kaart/planner, losse detector, besturing met testklok en MQTT-clientvervanger. Tests onder CPython importeren geen machine.
Brokerproef	Echte broker met afgescheiden testtopics. Test berichtenverlies, dubbele berichten, revisies en opnieuw verbinden.
Fysieke proef	T11, T12, T14-T18 op campus. Vermeld firmwareversie, voedingsspanning, kalibratie en werkelijke rijrichting.
Vrijgave	Alle criteria akkoord; afwijkingen opgelost of expliciet met docent/team afgesproken. Geen simulatoruitslag als hardwarebewijs gebruiken.

Minimaal testverslag

```
Test-ID / datum / uitvoerder:
Firmware- en kaartversie:
Startknoop en aankomstrichting:
Opdracht + command_id + mission_id + revisie:
Voeding / afstelling / serviceprofiel:
Verwacht / werkelijk resultaat:
Bewijs (log, video, screenshot):
Geslaagd of mislukt / afwijking / hertest:
```

Wat voor deze PDF al is gecontroleerd

Het documentmodel bevat 27 punten, 32 verbindingen en 112 toegestane lokale overgangen. Een documentcontrole vergeleek 1728 richtingbewuste start/doelgevallen met een onafhankelijke zoekmethode. Ook 325 geordende attractieselecties met depotterugkeer zijn op aansluiting gecontroleerd.

Dit controleert uitsluitend de voorgestelde kaart en voorbeeldplanning voor het logische eindpuntprofiel. De vijf productieklassen en de 18 testscenario's zijn hiermee niet geimplementeerd of fysiek geslaagd verklaard.

23 Bouwvolgorde en extra stappen

Iedere fase eindigt met een klein werkend resultaat. Extra klassen alleen toevoegen wanneer een concrete verantwoordelijkheid te groot wordt.

Stap	Resultaat / klaar als
1. Kaart afspreken	Foto en baan gecontroleerd; serviceplaatsen en start-/depotmethode gekozen. Open punten op pagina 24 invullen.
2. Kaart + planner	ParcoursKaart en RoutePlanner op PC; T01-T05 groen voor gekozen kaartversie.
3. Logische weergave	Kernrit en andere doelvolgorden stap voor stap zichtbaar. Correcte service-/eindvolgorde.
4. Detector + ritbesturing	Bestaande functies aangesloten; T09-T16 op PC; geen blokkerende halte/zoeklus.
5. MQTT aansluiten	Bestaande module aangepast aan contract; T06-T08 en T17. Lokale testtopics gescheiden van echte robot.
6. Baanproeven	Afstellen met ruwe opnames, korte ritten en terugkeer. Daarna kernrit en alle attracties.
7. Opleveren	T18, acceptatielijst, testbewijs, README en terugvalversie beschikbaar.

Toegevoegd omdat deze stappen makkelijk worden vergeten

- Halte en vertrek exact vastleggen: getekend eindpunt en echte T-stop niet verwisselen.
- Opdrachtsemantiek afspreken: vervangen van resterende doelen, sessies, volgnummers en duplicaten.
- Plaatsinschatting zichtbaar maken: een gemist punt veroorzaakt onzekerheid, geen automatische voltooiing.
- Sensorreeksen en voedingsconditie vastleggen om foutgedrag te kunnen herhalen.
- Tweede rit na afronden testen: geen oude bezoeklijst, PID-historie of detectorblokkering meenemen.

Grenzen van deze versie

Geen meerdere voertuigen, automatische herlokalisatie overal, permanente ritdatabase, uitgebreid dashboard of motor-/batterijmodel. Beveilig broker met accounts en beperkte topics; een productiebeveiligingsontwerp valt buiten dit schoolprototype.

24 Open beslissingen en bronnen

Geen onbekende hardware-eigenschappen als vaststaand feit invullen.

Nog vastleggen	Verantwoordelijkheid / effect
Echte serviceknoop per attractie	Team op baan: doel, aankomst, richting en vertrek. Daarna kaartversie en planner opnieuw testen.
Depotbevestiging en startorientatie	Routecontext plus op baan gekozen herkenningsmethode; breed zwart alleen is niet voldoende.
Vijf sensorbits en polariteit	Werkende hardware controleren; ruwe bitpatronen normaliseren. Geen zesde sensor uit een logstring afleiden.
Pinnen en module-API's	robot_mqtt.py en objectdetectie.py ontbreken hier; eerst bestaande campusbestanden controleren.
Timing / voeding / obstakelgrens	Twee representatieve voedingscondities en veilige kalibratie met werkelijke opnames vastleggen.
Broker en beoordelingscriteria	Bestaande client/QoS/pollgedrag, topicnamen, docentafspraken en toleranties bevestigen.

Bronnen en herkomst

[B1] OASIS, MQTT Version 3.1.1, 29 oktober 2014. Gebruikt voor publish/subscribe, QoS en retained berichten. Het applicatiecontract in deze PDF is een eigen ontwerp.

Officiele MQTT-specificatie

[B2] MicroPython, documentatie time: ticks_ms/ticks_diff. De geraadpleegde latest-pagina betreft de ontwikkeltak; kies bij implementatie ook de documentatie van de werkelijk geinstalleerde firmwareversie.

Officiele MicroPython-tijddocumentatie

[B3] Door gebruiker aangeleverde bovenaanzichtfoto: codex-clipboard-122eb65c-5931-4b0c-a6df-f2d68a94a596.jpg. Alleen voor leesrichting gedraaid; de digitale topologie is een afgeleide ontwerpschets.

[B4] Lokale main.py bekeken op 03-11-2026: vaste rit, vijf sensoren, basis 0.35, T-halte en bestaande MQTT-/objectimports. Geen softwarewijziging of robotmeting uitgevoerd voor dit document.

`SHA-256 foto: 6e137e7f6de8af69ea04b4544f28d3be51083c9426dfb45ccfc2fadc69f28cf3`

`SHA-256 main.py: 53ba8333414ffa33338dae33dfc41efed7a1bef31d89092f170f0564a0f03031`

Documentversie 2.0. De compacte keuzes vervangen de uitgebreide architectuur uit versie 1.0 voor de eerste oplevering. Raadpleeg het eerdere document alleen voor mogelijke latere uitbreiding.